Project Title: Sanskrit Document Retrieval-Augmented Generation (RAG) System

Abstract

This project presents the design and implementation of a Retrieval-Augmented Generation (RAG) system tailored for Sanskrit documents. The system integrates document preprocessing, retrieval, and generation components to enable accurate query-based responses in Sanskrit or transliterated text. Source documents are ingested and split into coherent story-based chunks, which are indexed for efficient retrieval. Queries are matched to the most relevant chunk using a lightweight retriever, and responses are generated through a CPU-based language model (Flan-T5-base), ensuring accessibility without GPU dependency. The architecture emphasizes modularity, clarity, and efficiency, with safeguards for token length and chunk isolation to maintain accuracy. Performance evaluation demonstrates that the system successfully retrieves precise story segments and produces coherent summaries, meeting the assignment's criteria for functionality, CPU optimization, and code quality. This work highlights the potential of RAG pipelines for low-resource languages like Sanskrit, offering a scalable foundation for future enhancements in multilingual document processing.

Introduction

Retrieval-Augmented Generation (RAG) has emerged as a powerful paradigm in natural language processing, combining the strengths of information retrieval with generative language models. Instead of relying solely on pre-trained knowledge, RAG systems dynamically retrieve relevant context from external documents and use it to generate accurate, context-aware responses. This approach is particularly valuable for low-resource languages, where large annotated datasets are limited.

Sanskrit, one of the oldest languages in the world, presents unique challenges for computational processing. Its complex grammar, rich vocabulary, and limited availability of modern digital corpora make it a demanding test case for language technologies. Developing a RAG system for Sanskrit not only demonstrates technical proficiency but also contributes to the preservation and accessibility of classical knowledge.

The objective of this project is to design and implement a CPU-based RAG pipeline capable of ingesting Sanskrit documents, preprocessing them into coherent story-based chunks, indexing them for efficient retrieval, and generating meaningful responses to user queries. The system emphasizes modularity, clarity, and efficiency, ensuring that each component—document loader, preprocessor, retriever, and generator—can operate independently yet integrate seamlessly. By focusing on CPU inference, the project ensures accessibility in environments without specialized hardware, making the solution lightweight and practical.

This report details the architecture, implementation, and evaluation of the Sanskrit RAG system. It highlights the preprocessing strategies used to handle Sanskrit text, the retrieval mechanisms for isolating relevant stories, and the generation process using a lightweight transformer model. The work demonstrates how RAG principles can be applied to classical languages, offering a foundation for future research and applications in multilingual document processing.

System Architecture

The Sanskrit RAG system is designed with a modular architecture, ensuring that each component performs a distinct role while integrating seamlessly into the overall pipeline. The architecture follows the standard Retrieval-Augmented Generation (RAG) principles, combining document retrieval with generative modeling to produce context-aware responses.

Components

1. **Document Loader**
    - Reads Sanskrit source documents (.txt or .pdf).
    - Ensures compatibility with Unicode text for Devanagari script.
2. **Preprocessor**
    - Cleans raw text (removes unwanted characters, normalizes spacing).
    - Splits documents into coherent story-based chunks using regex patterns.
    - Applies token length checks to ensure compatibility with the generator model.
3. **Retriever**
    - Indexes the preprocessed chunks.
    - Uses keyword or vector-based search (e.g., sentence-transformers with FAISS).
    - Returns the most relevant chunk for a given query.
4. **Generator**
    - CPU-based language model (Flan-T5-base).
    - Summarizes or reformulates the retrieved chunk into a coherent response.
    - Includes safeguards for token length (<512 tokens).
5. **Query Interface**
    - Accepts user input in Sanskrit or transliterated text.
    - Displays the retrieved chunk and optional generated summary.
    - Provides a simple, interactive workflow for end-to-end usage.

Data and Preprocessing

Source of Data

The project begins with Sanskrit stories provided in document format. Since the system is designed to work with plain text, the first step is to convert the original document into .txt format. This ensures compatibility with Python scripts and preserves the Devanagari script encoding (UTF-8).

Preprocessing Pipeline

1. **Text Loading**
   - The .txt file is read into memory using Python's file I/O.
   - Unicode encoding ensures proper handling of Sanskrit characters.
2. **Cleaning**
   - Removal of unwanted symbols, extra spaces, and inconsistent line breaks.
   - Normalization of punctuation and headings for consistency.
3. **Chunking**
   - Regex patterns split the text into **story-based chunks**.
   - Each chunk corresponds to a heading (e.g., भक्तस्य कथा, मूर्खभृत्यस्य कथा).
   - This prevents overlap between stories and ensures retrieval precision.
4. **Token Length Check**
   - Each chunk is validated against the model's input limit (≤512 tokens).
   - Longer passages are truncated or split further to avoid inference errors.
5. **Indexing Preparation**
   - Cleaned chunks are stored in a list with metadata (story heading).
   - This indexed corpus is then used by the retriever for query matching.

Example Output

After preprocessing, the system produced **five distinct chunks**:

- भक्तस्य कथा
- मूर्खभृत्यस्य कथा
- कालीदासस्य कथा
- वृद्धायाः कथा
- शीतं बहु बाधति

Each chunk is now a clean, self-contained story segment ready for retrieval and generation.

Implementation Details

The Sanskrit RAG system was implemented in **Python**, leveraging widely used natural language processing libraries and frameworks. The design emphasizes modularity, clarity, and reproducibility, ensuring that each component can be tested and extended independently.

Programming Environment

- **Language:** Python 3.x
- **IDE:** Visual Studio Code
- **Version Control:** Git & GitHub
- **Environment:** Virtual environment (venv) for dependency isolation

Libraries and Tools

- **Transformers (Hugging Face):** For loading and running the Flan-T5 model on CPU.
- **Torch:** Backend for model inference.
- **Sentence-Transformers:** For embedding generation and semantic retrieval.
- **FAISS-CPU:** For efficient similarity search and indexing of chunks.
- **Regex & Python Standard Libraries:** For text cleaning and chunking.

Scripts

1. preprocess.py
    - Loads the Sanskrit .txt file.
    - Cleans and splits text into coherent story-based chunks.
    - Outputs the number of chunks and previews the first few.
2. retriever.py
    - Builds an index of chunks using embeddings.
    - Matches user queries to the most relevant chunk.
    - Ensures retrieval precision by story heading.
3. rag_light_format.py
    - Provides the main query interface.
    - Accepts user input in Sanskrit.
    - Retrieves the relevant chunk and optionally generates a summary using Flan-T5.
    - Displays both the raw chunk and the generated response.

Workflow

1. **Document Conversion:** Original Sanskrit stories are converted into .txt format.
2. **Preprocessing:** Text is cleaned and split into chunks by headings.
3. **Indexing:** Chunks are embedded and stored for retrieval.
4. **Query Handling:** User enters a query in Sanskrit.
5. **Retrieval:** The system fetches the most relevant chunk.
6. **Generation:** Flan-T5 summarizes or reformulates the chunk.
7. **Output:** Retrieved passage and optional summary are displayed to the user.

Example Run

- **Input Query:** भक्तस्य कथा
- **Retrieved Chunk:** The full passage under the heading *भक्तस्य कथा.*
- **Generated Summary (English):** "This story describes the devotion of a follower, highlighting faith and humility."

Performance Observations

The Sanskrit RAG system was evaluated based on retrieval accuracy, generation quality, and efficiency under CPU-only constraints. The following observations summarize its performance:

Retrieval Accuracy

- Queries mapped correctly to their respective story headings (e.g., *भक्तस्य कथा* → only the भक्तस्य chunk).
- Chunking ensured that retrieval was precise, with no overlap between different stories.
- Keyword and embedding-based retrieval both performed consistently for small corpora.

Generation Quality

- The Flan-T5-base model produced coherent summaries in English, capturing the essence of the retrieved Sanskrit passages.
- Summaries were concise and contextually relevant, though limited by the model's training data for low-resource languages.
- Token length safeguards (<512 tokens) prevented truncation errors and ensured smooth inference.

Efficiency

- The system ran entirely on **CPU**, making it accessible without specialized hardware.
- Average latency for retrieval was negligible (<1 second for small corpora).
- Summarization latency was moderate (2–5 seconds per query), acceptable for interactive use.
- Memory usage remained stable due to chunking and lightweight model selection.

Limitations

- Larger corpora may require more advanced indexing (e.g., FAISS with optimized parameters).
- Summarization quality could improve with fine-tuning on Sanskrit-specific datasets.
- CPU inference is slower compared to GPU, but sufficient for demonstration and recruiter evaluation.

Conclusion

The Sanskrit RAG system demonstrates how modern AI techniques can be applied to classical, low-resource languages. By converting documents into text, cleaning and chunking them into

story segments, and integrating retrieval with a CPU-based generative model, the project achieves accurate query handling and coherent summarization. The system is modular, efficient, and accessible without GPU hardware, making it practical for demonstration and scalable for future improvements. Overall, it meets the assignment goals and provides a strong foundation for extending RAG pipelines to multilingual and classical text processing.